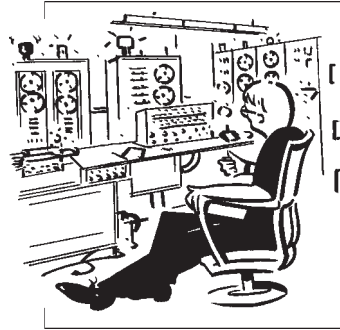


CHAPTER 27

overview of numerical methods



The aim of this Chapter...

...is to introduce the main numerical methods used in quantitative finance, and to propose a program of study.

In this Chapter...

- finite-difference methods
- Monte Carlo simulation
- numerical integration
- programs of study

27.1 INTRODUCTION

We are in the home straight now, only numerical methods separate us from the appendices and bibliography. This chapter, before we get to the meat of the numerical methods, is to put these techniques into context, to explain what they are used for, why, and give you some initial thoughts on how they are implemented. I'll also explain how efficient these techniques are in terms of computational time, and suggest a program of self study to help you build up your experience of implementation.

27.2 FINITE-DIFFERENCE METHODS

Finite-difference methods are designed for finding numerical solutions of differential equations. Since we work with a mesh, not unlike the binomial method, we will find the contract value at all points in stock price-time space. In quantitative finance that differential equation is almost always of diffusion or parabolic type, as explained. The only real differences between the partial differential equations are the following:

- number of dimensions;
- functional form of coefficients;
- boundary/final conditions;
- decision features;
- linear or non-linear.

Number of dimensions: Is the contract an option on a single underlying or many? Is there any strong path dependence in the payoff? Answers to these questions will determine the number of dimensions in the problem. At the very least we will have two dimensions: S or r , and t . Finite-difference methods cope extremely well with smaller number of dimensions, up to four, say. Above that they get rather time consuming.

Functional form of coefficients: The main difference between an equity option problem and a single-factor interest rate option problem is in the functional form of the drift rate and the volatility. These appear in the governing partial differential equations as coefficients. The standard model for equities is the lognormal model, but there are many more 'standard' models in fixed income. Does this matter? No, not if you are solving the equations numerically, only if you are trying to find a closed-form solution in which case the simpler the coefficients the more likely you are to find a closed-form solution. When we look at the details of the finite-difference methods we won't be assuming any particular functional forms at all.

Boundary/final conditions: In a numerical scheme the difference between a call and a put is in the final condition. You tell the finite-difference scheme how to start. And in finite-difference schemes in finance we start, strangely, at expiration and work towards the present. Boundary conditions are where we tell the scheme about things like knock-out barriers. When we write our code we'd like it to be as general and reusable as possible. That means writing it so that it doesn't have to be changed too much in going from one

contract or model to another. So we might put things like final conditions in some external function, to be changed easily.

Decision features: Early exercise, installment premiums, chooser features, are all examples of embedded decisions seen in exotic contracts. Coping with these numerically is quite straightforward using finite-difference methods, making these numerical techniques the natural ones for such contracts. The difference between a European and an American option is about three lines of code in a finite-difference program and less than a minute's work.

Linear or non-linear: Almost all finance models are linear, so that you can solve for a portfolio of options by solving each contract at a time and adding. Some more modern models are non-linear. Linear or non-linear doesn't make that much difference when you are solving by finite-difference methods. So choosing this method gives you a lot of flexibility in the type of model you can use.

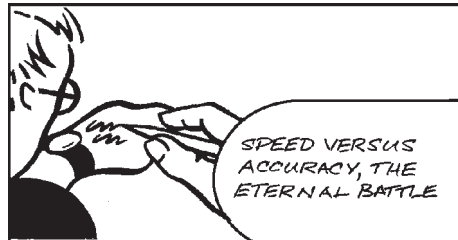
27.2.1 Efficiency

Finite differences are very good at coping with low dimensions, and are the method of choice if you have a contract with embedded decisions. They are excellent for non-linear differential equations.

Here I will just quote an expression for the calculation time of a typical finite-difference scheme used to price a portfolio of options. The time taken to price an option, and calculate the sensitivities to underlying(s) and time, will be

$$O\left(M\epsilon^{-1-d/2}\right),$$

where M is the number of options in the portfolio and we want an accuracy of ϵ , and d is the number of dimensions other than time.¹



27.2.2 Program of study

If you are new to numerics and you really want to study them to implement the models I describe then you need a program of study. Here I make some suggestions for how you should approach the numerical methods you'll be seeing.

- **Explicit method/European calls, puts and binaries:** To get started you should learn the explicit method as applied to the Black–Scholes equation for a European option. This is very easy to program and you won't make many mistakes.
- **Explicit method/American calls, puts and binaries:** Not much harder is the application of the explicit method to American options.

¹ So if we have a non-path-dependent option on a single underlying then $d = 1$.

- **Crank–Nicolson/European calls, puts and binaries:** Once you’ve got the explicit method under your belt you should learn the Crank–Nicolson implicit method. This is harder to program, but you will get a better accuracy. See *PWOQF2* for details concerning this numerical method and many of those below.
- **Crank–Nicolson/American calls, puts and binaries:** There’s not much more effort involved in pricing American-style options than in the pricing of European-style options.
- **Explicit method/path-dependent options:** By now you’ll be quite sophisticated and it’s time to price a path-dependent contract. Start with an Asian option with discrete sampling, and then try a continuously sampled Asian. Finally, try your hand at lookbacks.
- **Interest rate products:** Repeat the above program for non-path-dependent and then path-dependent interest rate products. First, price caps and floors and then go on to the index amortizing rate swap.
- **Two-factor explicit:** To get started on two-factor problems price a convertible bond using an explicit method, with both the stock and the spot interest rate being stochastic. Again, see *PWOQF2* for details.
- **Two-factor implicit:** The final stage is to implement the implicit two-factor method as applied to the convertible bond.

27.3 MONTE CARLO METHODS

Monte Carlo methods simulate the random behavior underlying the financial models. So, in a sense they get right to the heart of the problem. Always remember, though, that when pricing you must simulate the risk-neutral random walk(s), the value of a contract is then the expected present value of all cashflows. When implementing a Monte Carlo method look out for the following:

- number of dimensions;
- functional form of coefficients;
- boundary/final conditions;
- decision features;
- linear or non-linear;

again!

Number of dimensions: For each random factor you will have to simulate a time series. It will obviously take longer to do this, but the time will only be proportional to number of factors, which isn’t so bad. This makes Monte Carlo methods ideal for higher dimensions when the finite-difference methods start to crawl.

Functional form of coefficients: As with the finite-difference methods it doesn’t matter too much what the drift and volatility functions are in practice, since you won’t be looking for closed-form solutions.

Boundary/final conditions: These play a very similar role to finite differences. The final condition is the payoff function and the boundary conditions are where we implement trigger levels, etc.

Decision features: When you have a contract with embedded decisions the Monte Carlo method becomes cumbersome. This is easily the main drawback of simulation methods. You'll see that when we use the Monte Carlo method we are only finding the option value at today's stock price and time. But to correctly price an American option, say, we need to know what the option value *would be* at every point in stock price-time space. We don't typically find this as part of the Monte Carlo solution.

Linear or non-linear: Simulation methods also cope poorly with non-linear models. Some models just don't have a useful interpretation in terms of probabilities and expectations so you wouldn't expect them to be amenable to solution by methods based on random simulations.

27.3.1 Efficiency

If we want an accuracy of ϵ and we have d underlyings then the calculation time is

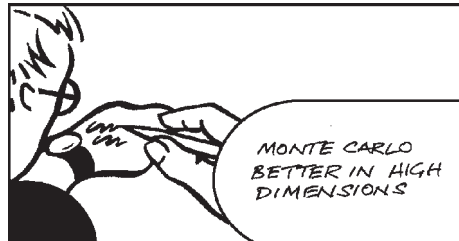
$$O(d\epsilon^{-3}).$$

It will take longer to price the greeks, but, on the positive side, we can price many options at the same time.

27.3.2 Program of study

Here is a program of study for the Monte Carlo path-simulation methods.

- **European calls, puts and binaries on a single equity:** Simulate a single stock path, the payoff for an option, or even a portfolio of options, calculate the expected payoff and present value to price the contract.
- **Path-dependent option on a single equity:** Price a barrier, Asian, lookback, etc.
- **Options on many stocks:** Price a multi-asset contract by simulating correlated random walks. You'll see how time taken varies with number of dimensions.
- **Interest rate derivatives, spot rate model:** This is not that much harder than equities. Just remember to present value along each realized path of rates *before* taking the expectation across all paths.
- **HJM model:** Slightly more ambitious is the HJM interest rate model. Use a single factor, then two factors, etc.
- **BGM model:** A discrete version of HJM.



27.4 NUMERICAL INTEGRATION

Occasionally one can write down the solution of an option-pricing problem in the form of a multiple integral. This is because you can interpret the option value as an expectation of a payoff, and an expectation of the payoff is mathematically just the integral of the product of that payoff function and a probability density function. This is only possible in special cases. The option has to be European, the underlying stochastic differential equation must be explicitly integrable (so the lognormal random walk is perfect for this) and the payoff shouldn't usually be path dependent. So if this is possible then pricing is easy ... you have a formula. The only difficulty comes in turning this formula into a number. And that's the subject of numerical integration or quadrature. Look out for the following:

- Can you write down the value of an option as an integral?

That's it in a nutshell.



27.4.1 Efficiency

There are several numerical quadrature methods we'll describe in Chapter 30. But the two most common are based on random number generation again. One uses Normally distributed numbers and the other uses what are called low-discrepancy sequences. The low-discrepancy numbers are clever in that they appear

superficially to be random but don't have the inevitable clustering that truly random numbers have.

Using the simple Normal numbers, if you want an accuracy of ϵ and you are pricing M options the time taken will be

$$O(M\epsilon^{-2}).$$

If you use the low-discrepancy numbers the time taken will be

$$O(M\epsilon^{-1}).$$

You can see that this method is very fast, unfortunately it isn't often applicable.

27.4.2 Program of study

Here is a program of study for the numerical quadrature methods.

- **European calls, puts and binaries on a single equity using Normal numbers:** Very simple. You will be evaluating a single integral.
- **European calls, puts and binaries on several underlying lognormal equities, using Normal numbers:** Very simple again. You will be evaluating a multiple integral.

- **Arbitrary European, non-path-dependent payoff, on several underlying lognormal equities, using Normal numbers:** You'll only have to change a single function.
- **Arbitrary European, non-path-dependent payoff, on several underlying lognormal equities, using low-discrepancy numbers:** Just change the source of the random numbers in the previous code.

27.5 SUMMARY

Subject	FD	MC	Quad.
Low dimensions	Good	Inefficient	Good
High dimensions	Slow	Excellent	Good
Path dependent	Depends	Excellent	Not good
Greeks	Excellent	Not good	Excellent
Portfolio	Inefficient	Very good	Very good
Decisions	Excellent	Poor	V. poor
Non-linear	Excellent	Poor	V. poor

If you get to the end of this program successfully then you will have reached a very high level of sophistication.

FURTHER READING

- Finite differences and Monte Carlo aren't the only numerical methods we can use. See Topper (2005) for a description of the finite-element method, a technique commonly used in the hard sciences and now finding its way into finance.

